

ALE 課程 5：Transaction Data 觸發機制

Lecture

ale04 示範了 Master Data 的自動化觸發：改一個欄位、開三層開關、跑 **BD21**，全程不用寫一行 ABAP。但 ale01 的對照表已經提過，**Transaction Data**（銷售訂單、發票、交貨單.....）沒有 **Change Pointer** 這種機制——不是 SAP 忘了做，是設計哲學上不適用：Change Pointer 回答的問題是「這筆主資料的哪個欄位被改了」，但一張銷售訂單從建立到出貨到開發票，中間有無數次欄位異動（改交期、改數量、改地址.....），如果每次異動都觸發分送，會產生大量沒意義的中間狀態通知。Transaction Data 真正在乎的是**特定的業務事件**（訂單確認、出貨完成、發票開立），不是任意欄位異動。

這題要講清楚 Transaction Data 兩種完全不同的觸發手段，並在 ale04 已經建立的「自動化 vs. 手動」對照基礎上，再加一個維度。

手段一：Output Determination（**NAST** / 條件技術）——「半自動」

很多有商業文件性質的交易資料（銷售訂單、交貨單、計費文件）天生就掛了 SAP 的**輸出決定（Output Determination）**框架——這跟你們可能在別的課程聽過的**定價條件技術（Condition Technique）**是同一套底層機制，只是應用在「這張單據該不該產生一個輸出（列印、傳真、EDI.....）」而不是「這張單據該賣多少錢」：

- **Output Type**（輸出類型，如 **RD04 = Invoice**、**BA00 = Order Confirmation**）：定義「這是哪一種輸出」，在 **NACE** 交易碼維護，每個 Output Type 都有 **Processing Routines**——指定不同的 **Transmission Medium**（傳輸媒介）分別該怎麼處理：媒介 **1** = 列印（走 Smartform，你們在 Smartform 課程學過）、媒介 **6** = EDI（走 ALE，這是本課關心的路徑）
- **Condition Record**（條件記錄）：業務人員 / Customizing 人員維護「什麼情況下要觸發哪個 Output Type」（例如「客戶 X 的所有發票都要用 EDI 傳輸媒介」），跟定價條件記錄的維護邏輯一模一樣
- **NAST**：每次單據存檔時，系統依照條件記錄判斷該不該產生輸出，如果該產生，就在 **NAST** 表插入一筆「待處理」的輸出記錄——**這張表的角色，就是 Transaction Data 版本的 Change Pointer**：一筆代辦事項，記錄「這張單據、這個輸出類型，還沒被處理」
- **處理**：跟 **BD21 / RBDMIDOC** 對應的角色是標準報表 **RSNAST00**（通常排程執行，或存檔當下就「Send Immediately」立即處理），依照 Output Type 設定的媒介，分別轉呼叫列印邏輯或 ALE 分送邏輯（媒介 **6** 的處理邏輯最終一樣會呼叫組 IDoc、送出的標準 ALE API）

這代表 **Output Determination** 是「半自動」：業務事件本身（單據存檔）是系統原生就會觸發判斷的時機點，你不需要自己找掛勾點；但要不要真的產生輸出、產生哪一種，是**條件記錄**決定的，等於是「觸發時機框架幫你抓好了，要不要動作你自己設定」。

手段二：手動 / Enhancement 觸發——「全手動」

但不是所有交易資料都有 Output Determination 這個框架可以掛。ale01 情境二就是這種案例：**MIRO（Invoice Verification）**完全沒有原生的輸出決定機制——它是財務/採購流程的一環，不是「要開一張單子給客戶」性質的文件，SAP 沒有內建「這張憑證該產生什麼輸出」的判斷框架。

這種情況下，觸發時機要靠你自己找：

- 用 **BTE (Business Transaction Event · FIBF)** 或 **Enhancement** (Enhancement 課程 en02 / en04 教過的技巧：Classic User-Exit / Explicit Enhancement Point 掛在單據存檔的處理邏輯裡)
- 掛勾點裡明確呼叫組 IDoc、送出的邏輯——最常見的作法是呼叫標準 ALE 分送 API **MASTER_IDOC_DISTRIBUTE** (跟 Change Pointer / Output Determination 背後最終呼叫的其實是同一支底層 API，只是這次要你自己決定「什麼時候呼叫它、傳什麼資料進去」)

這是「全手動」：連「該不該判斷」這個框架都沒有，觸發時機跟該不該送兩件事都要自己設計——ale09 (期末案例一) 就是完整實作這個路線的地方。

三種觸發機制總表——把 ale04 跟這題合起來看

	Change Pointer (ale04)	Output Determination (本題)	手動 / Enhancement (本題)
適用資料	Master Data	有原生輸出框架的 Transaction Data (銷售/計費/交貨)	沒有原生輸出框架的 Transaction Data (如 MIRO)
觸發時機判斷	系統自動 (欄位層級白名單)	系統自動 (單據存檔時機已內建)	完全自己設計 (Enhancement 掛勾點)
要不要送	BD52 欄位開關	條件記錄	自己寫程式邏輯判斷
待處理清單表	Change Pointer (BDCP 系列)	NAST	無 (即時呼叫，通常不留待處理佇列)
執行報表	BD21 / RBDMIDOC	RSNAST00 (或存檔即時處理)	無，掛勾點直接呼叫 API
本課程對應	ale04	本題觀念 + ale10 期末案例二深入實作	本題觀念 + ale07/ale09 深入實作

學習目標

- 能講出 Output Determination (NAST / 條件技術) 跟 Change Pointer 在「自動化程度」上的差異：都是系統原生會判斷觸發時機，但一個是欄位層級白名單、一個是條件記錄
- 能講出為什麼有些交易資料 (如 MIRO) 沒有原生輸出框架可用，這種情況下觸發時機要怎麼自己設計
- 能完成三種觸發機制 (Change Pointer / Output Determination / 手動-Enhancement) 的完整對照，講出各自的「待處理清單表」與「執行報表/API」
- 能在 NACE 找到一個 Output Type，看懂 Processing Routines 裡不同 Transmission Medium 各自對應的處理方式
- 能在既有商業文件上，用畫面上的 Output/Messages 頁籤看到 Output Determination 實際留下的處理記錄

事前準備

本題不會新增任何 Customizing 設定或新建物件，全部是唯讀觀察 (既有標準 Output Type 設定 + 既有歷史單據的輸出記錄)，比 ale02 ~ 04 風險更低。真正的 Output Determination 端對端建置留給 ale10 (那題會真的設定條件記錄、走完 Billing→INVOIC 全流程)。

題目需求

步驟 1：NACE 觀察 Output Type 的 Processing Routines

1. 交易碼 **NACE**
2. **Application** 選 **V3** (Billing , 計費文件 ; ⚠ **V1** = Sales 、 **V2** = Shipping 、 **V3** = Billing , 2026-09-21 用 **NAST** 實測 : **RD04** 的 **KAPPL** 是 **V3** , **V2** 底下是 **LD00** 交貨單輸出)
3. 工具列 **Output Types** → 找到 **RD04** (Invoice) → 選取後點 **Processing Routines**
4. 觀察畫面列出的每一列 : **Transmission Medium** 欄位有哪些選項 (留意有沒有一列的媒介是 EDI / External Send 這一類 , 跟其他列如「1 = Print」使用的欄位 (**Form** / **Smartform**) 是否不同——EDI 那一列通常改用 **Program / FORM Routine** 或直接是 **Function Module** 這類欄位 , 不會有列印用的 Form 名稱)
5. 記下你實際看到的畫面內容 (欄位名稱可能因版本略有差異 , 照你看到的實際內容記錄 , 不用照抄本題文字)

步驟 2：找一張既有計費文件，看它的 Output 記錄

1. 交易碼 **VF03** (Display Billing Document) , 輸入任意一張既有的計費文件號碼 (如果不知道號碼 , 可以用 **VF05** 先查一張出來)
2. 進入文件後 , 選單 **Goto** → **Header** → **Output** (或工具列類似按鈕 , 畫面標籤可能是 **Messages** 或 **Output**)
3. 觀察清單 : 這張單據存檔時 , 系統判斷出了哪些 Output Type ? 每一筆的 **Processing Status** (通常會有圖示或文字 , 如「已處理」 / 「未處理」) 是什麼 ?
4. 如果有任何一筆狀態是「已處理」 , 點進去看它的 **Processing Log** , 確認實際是走列印還是走別的媒介

步驟 3：對照三種機制的白板練習（不需要在系統操作，用文字整理即可）

針對下面兩個新情境，判斷該用哪一種觸發機制 (Change Pointer / Output Determination / 手動-Enhancement) , 並說明理由：

1. 情境 A：集團內部規則要求，任何一張交貨單 (Delivery) 出貨完成後，都要通知另一個系統更新庫存預估
2. 情境 B：某個自訂的內部簽核單據 (純 Z 程式，沒有掛任何標準框架) 核准通過後，要通知另一個系統

參考答案

題目需求 3 參考答案

- **情境 A：Output Determination**——交貨單 (Delivery) 是有原生輸出框架的標準商業文件 (**LD00** 之類的標準 Output Type 本來就存在) , 出貨完成是單據存檔會觸發判斷的天然時機點 , 只要設定條件記錄即可 , 不需要自己找掛勾點。
- **情境 B：手動 / Enhancement 觸發**——純自訂的 Z 程式沒有任何標準框架可以掛 (沒有 Output Determination、也不是主資料沒有 Change Pointer 可用) , 核准通過這個事件要靠開發者自己在該程式的「核准」邏輯裡明確呼叫 ALE 分送 API , 這跟 **MIRO** 的情況是同一類。

思考題

1. Output Determination 的條件記錄可以設定成「星期一到五都送、週末不送」這種複雜規則，Change Pointer 的 **BD52** 卻只能是「這個欄位要不要追蹤」的簡單開關，沒有類似的條件式判斷。你覺得為什麼 Master Data 的觸發機制設計得比 Transaction Data 簡單？（提示：想想 Master Data 通常只有一份「現在的狀態」，Transaction Data 卻有複雜的業務流程階段——這對「要不要送」這個判斷的複雜度需求有什麼影響）
2. 手動 / Enhancement 觸發的表格裡「待處理清單表」欄位是「無」——代表它是**即時**呼叫 API、沒有累積代辦清單這個中間層。這樣的設計對「如果 ALE 分送當下失敗了，要怎麼重試」這件事，跟 Change Pointer / **NAST** 這種「先留一筆待處理記錄」的設計相比，各自的優劣是什麼？（提示：想想 **BD21** / **RSNAST00** 可以重複執行去「掃一遍還沒處理的」，手動呼叫模式如果失敗了，誰會知道要重試）

答案

本題全程唯讀觀察標準 SAP 既有設定與既有歷史單據，沒有新增任何 Customizing 或物件，也沒有可快照的 ABAP 原始碼。完成步驟 1 / 2 後，把你在 **NACE** / **VF03** 實際看到的畫面內容（欄位名稱、Processing Status 顯示方式）回報給我，我會核對是否跟本題 Lecture 的說明一致，如果你的版本畫面用詞不同，我會更新這份講義的措辭。步驟 3 的白板練習答案見上方「參考答案」。